Intro-Intermediate to Python

- **Introduction to Python**: Python is a versatile, open-source programming language created by Guido Van Rossum. It's widely used for various applications, including data science.
- **Interactive Python (IPython)**: You experimented with IPython, an enhanced version of the standard Python shell, useful for immediate feedback and experimentation.
- **Python Scripts**: You learned that Python scripts are text files with a `.py` extension. They allow you to execute multiple Python commands in sequence.
- **Basic Calculations**: You practiced using Python for basic arithmetic operations:

```
print(4 + 5)  # Addition

print(5 - 5)  # Subtraction

print(3 * 5)  # Multiplication

print(10 / 2)  # Division
```

Matplotlib

- **Introduction to Matplotlib**: You discovered that Matplotlib is essential for creating insightful and visually appealing plots. The subpackage `pyplot` is commonly imported as `plt`.

Line Plots: You learned how to create a line plot to visualize data trends over time. For example:

```
import matplotlib.pyplot as plt
plt.plot(year, pop)
plt.show()
```

- This code plots the `year` on the x-axis and `pop` (population) on the y-axis, and displays the plot.

Scatter Plots: You explored scatter plots, which are useful for showing the relationship between two variables. For instance:

```
plt.scatter(gdp_cap, life_exp)
plt.xscale('log')
plt.show()
```

- This code creates a scatter plot with `gdp_cap` on the x-axis and `life_exp` on the y-axis, using a logarithmic scale for the x-axis.
- **Interpreting Plots**: You practiced interpreting plots to extract meaningful insights, such as predicting future population growth or understanding the correlation between GDP and life expectancy.

- **Histogram Basics**: A histogram divides data into bins and counts the number of data points in each bin. The height of each bar represents the count of data points within that bin.
- **Creating Histograms with Matplotlib**: You used the `hist` function from the `pyplot` package to create histograms. The `x` argument specifies the data, and the `bins` argument determines the number of bins.
- **Default and Custom Bins**: If you don't specify the `bins` argument, it defaults to 10. You experimented with different numbers of bins to see how it affects the histogram's detail and clarity.

Here's a simple example to create a histogram in Python:

```
import matplotlib.pyplot as plt

# Sample data

data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5]

# Create histogram with 3 bins

plt.hist(data, bins=3)

# Display the histogram

plt.show()
```

- **Comparing Histograms**: You compared histograms for different datasets to observe changes over time, such as life expectancy data from 1950 and 2007.
- **Choosing the Right Plot**: You learned that histograms are ideal for visualizing distributions, while scatter plots are better for assessing correlations between two variables.
- **Labeling Axes and Titles**: You used `xlabel()`, `ylabel()`, and `title()` to add descriptive labels and titles to your plots, making it clear what the data represents.

Adjusting Ticks: You customized the ticks on the axes using `yticks()` and `xticks()`, which helps in making the plot more readable. For example:
`plt.yticks([0, 2, 4, 6, 8, 10], ['0', '2B', '4B', '6B', '8B', '10B'])`

- **Scatter Plot Customizations**: You created a scatter plot with GDP per capita on the x-axis and life expectancy on the y-axis. You added axis labels and a title to this plot.
- **Tick Customization**: You learned how to replace tick values with more readable labels using `xticks()`.
- **Bubble Sizes**: You adjusted the size of the scatter plot bubbles to reflect population sizes by converting the population list to a NumPy array and scaling it.

- **Adding Colors**: You added colors to the scatter plot based on continent data and adjusted the opacity for better visualization.
- **Additional Customizations**: You added text annotations and gridlines to your plot for better clarity and readability.
- **Introduction to Dictionaries**: You discovered that dictionaries use curly brackets {} and store data in key-value pairs, making data retrieval straightforward.

Creating a Dictionary: You learned to create a dictionary by defining keys and their corresponding values. For example:
`world = {'Afghanistan': 30.55, 'Albania': 2.77, 'Algeria': 40}`

- **Accessing Values**: You practiced accessing values by using the key inside square brackets. For instance, `world['Albania']` returns `2.77`.
- **Efficiency**: You learned that dictionaries are not only intuitive but also efficient for data lookups, even with large datasets.
- **Accessing Values**: You can retrieve a value by using its key in square brackets. For example, `world['Albania']` returns the population of Albania.
- **Adding and Updating Entries**: You can add a new key-value pair or update an existing one using the same syntax. For example, `world['Sealand'] = 27` adds Sealand with a population of 27.
- **Removing Entries**: Use the `del` statement to remove a key-value pair. For example, `del(world['Sealand'])` removes Sealand from the dictionary.
- **Immutability of Keys**: Keys must be immutable types like strings, integers, or tuples. Mutable types like lists cannot be used as keys.

Here's a code snippet to illustrate adding and updating entries in a dictionary:

```
# Definition of dictionary

europe = {'spain': 'madrid', 'france': 'paris', 'germany': 'berlin', 'norway': 'oslo'}

# Add italy to europe

europe['italy'] = 'rome'

# Print out italy in europe

print('italy' in europe)  # Outputs: True

# Add poland to europe

europe['poland'] = 'warsaw'

# Print europe
```

```
print(europe)
```

Pandas

- **Pandas DataFrame**: A high-level data structure for storing tabular data, where rows represent observations and columns represent variables.

Creating DataFrames: You can create a DataFrame manually from a dictionary. The keys are column labels, and the values are lists of column data.

```
import pandas as pd

names = ['United States', 'Australia', 'Japan', 'India', 'Russia', 'Morocco', 'Egypt']

dr = [True, False, False, False, True, True, True]

cpc = [809, 731, 588, 18, 200, 70, 45]

my_dict = {'country': names, 'drives_right': dr, 'cars_per_cap': cpc}

cars = pd.DataFrame(my_dict)

print(cars)
```

- **Setting Row Labels**: You can set custom row labels using the `index` attribute of the DataFrame.

Importing Data from CSV: Instead of manually creating DataFrames, you can import data from CSV files using the `pd.read_csv()` function. You can specify which column to use as row labels with the `index_col` argument.

```
cars = pd.read_csv('cars.csv', index_col=0)

print(cars)
```

- **Square Brackets**:
    - Single square brackets ([]) return a Pandas Series.
    - Double square brackets ([[]]) return a Pandas DataFrame.

  Example:

  ```
  cars['country']  # Series

  cars[['country']]  # DataFrame
  ```

    -
- **Selecting Rows with Slicing**:
    - Use slices to select specific rows.

Example:
`cars[0:3]  # First three rows`

- loc **and** iloc:
  - loc is label-based, using row and column labels.
  - iloc is index-based, using integer positions.

  Example:
  `cars.loc['RU']  # Row for Russia using label`

  `cars.iloc[4]  # Row for Russia using index`

- **Combining Rows and Columns**:
  - Use loc and iloc to select specific rows and columns simultaneously.

  Example:
  `cars.loc[['RU', 'MOR'], ['country', 'drives_right']]  # Specific rows and columns`

- **Comparison Operators**: You discovered that these operators compare two values and return a Boolean indicating the relationship's truthfulness. For example, x > 5 checks if x is greater than 5.
- **Boolean Operators**: You learned about three key Boolean operators:
  - and: Returns True if both sides are True. For instance, True and False results in False.
  - or: Returns True if at least one side is True. For example, True or False results in True.
  - not: Negates the Boolean value it precedes. Thus, not True is False.
- **Combining Comparisons**: You practiced combining comparison operations with Boolean operators to form more complex conditions, like checking if a variable x is between two values with x > 5 and x < 15.

Additionally, you learned how to apply these concepts to NumPy arrays and encountered the specialized functions np.logical_and, np.logical_or, and np.logical_not for element-wise logical operations. For example, to find elements of an array my_house that are greater than 18.5 or smaller than 10, you used:

```
import numpy as np

my_house = np.array([18.0, 20.0, 10.75, 9.50])

print(np.logical_or(my_house > 18.5, my_house < 10))
```

If, else, elif

- if **statement**: Used to execute a block of code if a condition is true. For example, `if area < 9: print("small")` checks if the variable `area` is less than 9 and prints "small" if the condition is true.
- `else` **statement**: Follows an `if` statement and is executed if the `if` condition is false. It does not require a condition. For example, in the context of `if` statements, if `area` is not less than 9, the `else` block can print "large" as a default action.
- `elif` **(else if) statement**: Used after an `if` statement to check another condition if the previous condition was false. For example, `elif area < 12: print("medium")` can be used to check if `area` is less than 12 but only after the `if` condition has been found to be false.

Here's a code snippet illustrating these concepts:

```
area = 10.0

if area < 9:

    print("small")

elif area < 12:

    print("medium")

else:

print("large")
```

- **Selecting DataFrame columns**: You discovered how to extract a column from a DataFrame, which returns a pandas Series. This is done using square brackets and the column name, e.g., brics['area'].
- **Boolean indexing**: You learned to perform comparisons on DataFrame columns, generating Boolean Series. These Series can then be used to filter the DataFrame, selecting only rows where the condition is True. For example, to filter countries with an area greater than 8 million square kilometers, you used: is_huge = brics['area'] > 8.
- **Using Boolean Series for filtering**: You applied the Boolean Series as an index to the DataFrame to select only the rows of interest, like brics[is_huge].
- **Simplifying with one-liners**: You combined these steps into a one-liner, directly placing the comparison inside the square brackets used for indexing, e.g., brics[brics['area'] > 8].
- **Advanced Boolean operations**: You utilized np.logical_and() to perform element-wise logical operations, enabling more complex filters, such as selecting observations within a specific range.

Here's a snippet demonstrating how to filter for countries with an area greater than 8 million square kilometers:

```
import pandas as pd
# Assuming brics DataFrame is already defined
is_huge = brics['area'] > 8
huge_countries = brics[is_huge]
print(huge_countries)
```

- The syntax and structure of a while loop, which is similar to an if statement but continues to execute as long as the condition remains true.

An example of a while loop to numerically calculate a model's error and reduce it by a factor until it falls below a certain threshold. The key code snippet demonstrated this concept:

```
error = 50.0
while error > 1 :
  error = error / 4
  print(error)
```

- The importance of updating the loop condition within the loop to prevent infinite loops. You saw how failing to update the error variable within the loop could lead to a loop that never ends.
- How to use a while loop with conditionals to handle different scenarios, such as correcting an offset that could be positive or negative. You learned to increment or decrement the offset value appropriately to eventually meet the loop's exit condition.

Basic for **Loop Structure**: You saw how to iterate over a list using a simple for loop, printing each element. For example:

```
for height in fam:
  print(height)
```

- This loop goes through each item in the `fam` list, assigns it to the variable `height`, and prints it.

Using `enumerate()` **for Indexing**: To access both the index and the value of each item in a list, you used `enumerate()`. It modifies the loop to produce index-value pairs:

```
for index, area in enumerate(areas):
  print("room " + str(index) + ": " + str(area))
```

- This allows you to print both the index of the item and its value, making your output more informative.

Adjusting Indexes in Output: You learned to adjust the starting index in the output to be more intuitive (starting from 1 instead of 0) by modifying the index inside the print statement:
`for index, area in enumerate(areas):`

```
    print("room " + str(index + 1) + ": " + str(area))
```

-

Looping Over List of Lists: Finally, you explored iterating over a list of lists, where each sublist contains multiple pieces of data. You printed formatted strings that include both elements of each sublist, demonstrating a practical way to handle nested lists:
`for x in house:`

- `    print("the " + x[0] + " is " + str(x[1]) + " sqm")`

Dictionaries: You learned that to iterate over both keys and values in a dictionary, you should use the `.items()` method. This method allows you to access each key-value pair in the dictionary, enabling you to print or manipulate them. For example:
`for key, value in dictionary.items():`

```
    print(key, value)
```

- This approach is essential for when you need to work with both keys and values simultaneously.
- **NumPy Arrays**: You discovered that iterating over NumPy arrays requires different approaches depending on the array's dimensionality. For a 1D array, a simple for loop suffices. However, for 2D arrays or higher, you need to use the `np.nditer()` function to efficiently iterate over each element. This function is crucial for dealing with multi-dimensional data in data science tasks.
- **Practical Application**: Through exercises, you applied these concepts by iterating over a dictionary of European countries to print the capital of each country and looping over NumPy arrays to print each element, reinforcing your understanding of how to navigate these common data structures in Python.

Iterating over a DataFrame using `iterrows()` which provides the row label and data as a Pandas Series in each iteration. For example:
`for lab, row in cars.iterrows():`

```
    print(lab)

    print(row)
```

- Selecting specific data from each row using the column name, like `row['column_name']`, to print or manipulate data.

- Adding a new column to a DataFrame within a loop. You saw how to use `iterrows()` to iterate over each row and then use `loc` or direct assignment to add or modify a column based on data from other columns.

The efficiency of using `apply()` for element-wise operations on columns over using a for loop with `iterrows()`. For instance, converting a column to uppercase:
`cars["COUNTRY"] = cars["country"].apply(str.upper)`

This approach is not only more concise but also significantly faster for larger datasets

- **Random Number Generators (RNGs)**: You discovered how to use RNGs in Python with the `numpy` library, understanding the importance of pseudo-random numbers in simulations. For instance, generating a random float between 0 and 1 with `np.random.rand()`.
- **Setting a Seed**: You learned the significance of setting a random seed using `np.random.seed(123)`, ensuring reproducibility in your simulations. This allows for consistent results across runs.
- **Simulating Dice Rolls**: You simulated dice rolls using `np.random.randint(1,7)`, which is crucial for the game's logic where the outcome determines your next move.

Control Structures: You applied if-elif-else statements to simulate the decision-making process based on the dice roll. For example:

```
if dice <= 2:
    step = step - 1
elif dice <= 5:
    step = step + 1
else:
    step = step + np.random.randint(1,7)
```

- **Random Walk Basics**: You learned that a random walk is a succession of random steps, much like the path a molecule takes in a gas or liquid, or the fluctuation in a gambler's financial status.
- **Building a Random Walk in Python**: Through coding exercises, you discovered how to simulate a random walk using Python. This involved generating random steps with a dice roll and accumulating these steps to model a walk.
- **For Loops and Lists**: You practiced using `for` loops to iterate over a range of numbers, and how to use lists to store the outcomes of each step in the random walk.

- **Conditional Logic**: The exercises reinforced your understanding of `if-else` statements to determine the direction of each step based on the dice roll.
- **Improving the Simulation**: You enhanced the random walk simulation by ensuring the step value never goes below zero using the `max()` function, addressing the scenario where a step could theoretically take you to a negative position.
- **Visualization with Matplotlib**: Finally, you visualized the random walk using `matplotlib`, plotting the walk to see the path that was taken over 100 steps.

Here's a snippet from the lesson that encapsulates creating and plotting a random walk:

```
import numpy as np

import matplotlib.pyplot as plt



# Initialize random_walk

random_walk = [0]

for x in range(100):

    step = random_walk[-1]

    dice = np.random.randint(1, 7)

    if dice <= 2:

        step = max(0, step - 1)

    elif dice <= 5:

        step += 1

    else:

        step += np.random.randint(1, 7)

    random_walk.append(step)

# Plot random_walk

plt.plot(random_walk)

plt.show()
```